

Advanced computerarchitecture
Exercise 1 Report

(B-KUL-JPI30R)

Wout Van Steenbergen

February 18, 2024



Session Date: February 12, 2024

Instructor: Alex Marinšek

1 Introduction

During these lab sessions usability of CUDA is explored. It stands for Compute Unified Device Architecture, enables developers to use C/C++ like code to write software that can perform computational tasks on the GPU. This approach is particularly beneficial for tasks that can be executed in parallel, offering significant performance improvements over traditional CPU-based computation for certain applications. The first exercise asks for reversing an array using the parallel computing capabilities of NVIDIA GPUs.

The heart of the program is the flipArrayKernel function, attached with the __global__ declaration specifier. This specifier is unique to CUDA and indicates that the function is a kernel, which is a function that can be executed on the GPU by numerous threads in parallel. The kernel's purpose is to reverse an array. It achieves this by calculating an index for each thread and a corresponding flipped index . The expression "int index = blockIdx.x * blockDim.x + threadIdx.x;" is responsible for this calculation. The first part of the expression computes the starting index of the current block within the global thread indexing scheme. By multiplying the block's index along the x-axis (blockIdx.x) by the number of threads per block (blockDim.x), you get the offset that tells you how many threads precede the current block. Adding the thread's index within its block (threadIdx.x) to this offset gives the final global index of the thread. This design ensures that each thread is responsible for copying a single element from the input array to the correct position in the output array.

The Main function begins by defining the size of the array and allocating memory for both the input and output arrays on the host. Memory allocation on the device (GPU) mirrors this process, preparing space for the data that will be manipulated by the kernel. Host memory is directly accessible by the CPU, while device memory is directly accessible by the GPU. To use GPU for computations, data must be explicitly transferred from host memory to device memory, processed, and then transferred back if needed. In CUDA programs, developers must explicitly manage memory allocation, transfer, and deallocation in both host and device memory spaces. This involves using specific CUDA API calls (e.g., cudaMalloc, cudaMemcpy, cudaFree) to handle device memory and standard C/C++ memory management functions for host memory. The input array is then filled with random integers and copied from host to device memory.

Next the number of threads and blocks are chosen. For NVIDIA CPU's this number should lie between 32 and 1024. A block size of 256 threads is often near optimal for many applications. Since the number of threads is hardcoded the number of blocks must be calculated. Simply dividing n by threads (n / threads) would tell us how many full blocks of 256 threads are needed to cover n elements. However, if n is not an exact multiple of threads, this division will result in a shortfall, leaving some elements unprocessed. Adding threads - 1 to n before dividing ensures that this shortfall is accounted for by effectively rounding up the division. This results in the following: "int blocks = (n + threads -1) / threads;" Because of the rounding up a check needs to be added in the flipArrayKernel that prevents the index of the array to go out of range.

After the kernel completes its execution, the reversed array is copied back from the device to the host, allowing the host to access and check for any mistakes. If no mismatches are found throughout the entire array, indicating that the array has been successfully reversed, it prints a confirmation message.